In-Person

Genomics Compute Cluster

Scaling Up for High-Throughput Computing

Northwestern | INFORMATION TECHNOLOGY
RESEARCH COMPUTING AND DATA SERVICES

Start by opening up a script to modify…

1. log onto Quest
2. move into our allocation directory
3. make a personal folder if you don't already have one
4. copy `/projects/e32559/example_data/bwa.sh` to your personal folder
5. open your copy of `bwa.sh` with your editor of choice

Case-Study Being Covered: General

- There is a script or program that I need to run on Quest many times while *only* changing input files and/or input arguments.
  - Step 1: Optimally configure the script for a single job.
  - Step 2: Take that optimal configuration for a single job convert to it a job array to run it on many different input file and/or input arguments.

Case-Study Being Covered: Specific

- I have a genomics pipeline/program that I want to run on many fastq files.
  - Step 1: Optimally configure the script used for a single run of the program on a single pair of fastq files.
  - Step 2: Take that optimal configuration for a single job/run convert to a job array to do it on many many paired fastq files.

Goals - scaling up for high throughput

1. Use **variables** to remove hard-coding and increase generalizability
2. Use **job arrays** to launch multiple jobs with one script

Use **variables** to remove hard-coding and increase generalizability

- Hard-coding refers is when a specific value is written into the script. For example, this could be an exact value, or an absolute file path.
- Replace repetition and hard-coding with variables.
- Bash variables are set with "=" and referenced with $

```
export FOOD=pie
echo $FOOD
```

- Convention is capitals for variable names
- "export" is used to retain the variable in the environment for child processes

```
#!/bin/bash
#SBATCH --account=e32559
#SBATCH --partition=normal
#SBATCH --job-name=bwa_align
#SBATCH --time=10:00:00
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1
```

Requesting resources from the scheduler

```
module load bwa/0.7.17
```

Using specific bwa module

```
bwa mem -t 8 \
 /projects/e32559/example_data/oenotheraHarringtonii.softmasked.fasta \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R1_001.fastq.gz \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R2_001.fastq.gz \
 > TRIN_5690_11_S31_L001_aln-pe.sam
```

```
#!/bin/bash
#SBATCH --account=e32559
#SBATCH --partition=normal
#SBATCH --job-name=bwa_align
#SBATCH --time=10:00:00
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

bwa mem -t 8 \
 /projects/e32559/example_data/oenotheraHarringtonii.softmasked.fasta \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R1_001.fastq.gz \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R2_001.fastq.gz \
 > TRIN_5690_11_S31_L001_aln-pe.sam
```

The bwa mem command takes 5 arguments:

- threads
- genome
- fastq files (1+2)
- output file

What is hard-coded?

```
#!/bin/bash
#SBATCH --account=e32559
#SBATCH --partition=normal
#SBATCH --job-name=bwa_align
#SBATCH --time=10:00:00
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

bwa mem -t 8 \
 /projects/e32559/example_data/oenotheraHarringtonii.softmasked.fasta \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R1_001.fastq.gz \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R2_001.fastq.gz \
 > TRIN_5690_11_S31_L001_aln-pe.sam
```

The bwa mem command takes 5 arguments:

- threads
- genome
- fastq files (1+2)
- output file

All of the arguments.

```
#!/bin/bash
#SBATCH --account=e32559
#SBATCH --partition=normal
#SBATCH --job-name=bwa_align
#SBATCH --time=10:00:00
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

bwa mem -t 8 \
 /projects/e32559/example_data/oenotheraHarringtonii.softmasked.fasta \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R1_001.fastq.gz \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R2_001.fastq.gz \
 > TRIN_5690_11_S31_L001_aln-pe.sam
```

The bwa mem command takes 5 arguments:

- threads
- genome
- fastq files (1+2)
- output file

Where is there repetition?

```
#!/bin/bash
#SBATCH --account=e32559
#SBATCH --partition=normal
#SBATCH --job-name=bwa_align
#SBATCH --time=10:00:00
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

bwa mem -t 8 \
 /projects/e32559/example_data/oenotheraHarringtonii.softmasked.fasta \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R1_001.fastq.gz \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R2_001.fastq.gz \
 > TRIN_5690_11_S31_L001_aln-pe.sam
```

The bwa mem command takes 5 arguments:

- threads
- genome
- fastq files (1+2)
- output file

Where is there repetition?

```
#SBATCH --job-name=bwa_align
#SBATCH --time=10:00:00
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

export DIR=/projects/e32559/example_data

bwa mem -t 8 \
 $DIR/oenotheraHarringtonii.softmasked.fasta \
 $DIR/TRIN_5690_11_S31_L001_R1_001.fastq.gz \
 $DIR/TRIN_5690_11_S31_L001_R2_001.fastq.gz \
 > TRIN_5690_11_S31_L001_aln-pe.sam
```

The bwa mem command takes 5 arguments:

- threads
- genome
- fastq files (1+2)
- output file

```
#!/bin/bash
#SBATCH --account=e32559
#SBATCH --partition=normal
#SBATCH --job-name=bwa_align
#SBATCH --time=10:00:00
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

bwa mem -t 8 \
 /projects/e32559/example_data/oenotheraHarringtonii.softmasked.fasta \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R1_001.fastq.gz \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R2_001.fastq.gz \
 > TRIN_5690_11_S31_L001_aln-pe.sam
```

The bwa mem command takes 5 arguments:

- threads
- genome
- fastq files (1+2)
- output file

Where is there repetition?

```
#SBATCH --time=10:00:00
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

export DIR=/projects/e32559/example_data
export SAMPLE=TRIN_5690_11_S31_L001

bwa mem -t 8 \
  $DIR/oenotheraHarringtonii.softmasked.fasta \
  $DIR/$SAMPLE_R1_001.fastq.gz \
  $DIR/$SAMPLE_R2_001.fastq.gz \
  > $SAMPLE_aln-pe.sam
```

The bwa mem command takes 5 arguments:

- threads
- genome
- fastq files (1+2)
- output file

```
#!/bin/bash
#SBATCH --account=e32559
#SBATCH --partition=normal
#SBATCH --job-name=bwa_align
#SBATCH --time=10:00:00
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

bwa mem -t 8 \
 /projects/e32559/example_data/oenotheraHarringtonii.softmasked.fasta \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R1_001.fastq.gz \
 /projects/e32559/example_data/TRIN_5690_11_S31_L001_R2_001.fastq.gz \
 > TRIN_5690_11_S31_L001_aln-pe.sam
```

The bwa mem command takes 5 arguments:

- threads
- genome
- fastq files (1+2)
- output file

Where is there repetition?

```
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

export DIR=/projects/e32559/example_data
export SAMPLE=TRIN_5690_11_S31_L001

bwa mem -t $SLURM_NTASKS \
  $DIR/oenotheraHarringtonii.softmasked.fasta \
  $DIR/$SAMPLE_R1_001.fastq.gz \
  $DIR/$SAMPLE_R2_001.fastq.gz \
  > $SAMPLE_aln-pe.sam
```

The bwa mem command takes 5 arguments:

- threads
- genome
- fastq files (1+2)
- output file

```
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

export DIR=/projects/e32559/example_data
export SAMPLE=TRIN_5690_11_S31_L001

bwa mem -t $SLURM_NTASKS \
  $DIR/oenotheraHarringtonii.softmasked.fasta \
  $DIR/$SAMPLE_R1_001.fastq.gz \
  $DIR/$SAMPLE_R2_001.fastq.gz \
  > $SAMPLE_aln-pe.sam
```

The bwa mem command takes 5 arguments:

- threads
- genome
- fastq files (1+2)
- output file

```
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

export DIR=/projects/e32559/example_data
export SAMPLE=TRIN_5690_11_S31_L001
export GENOME=oenotheraHarringtonii.softmasked.fasta

bwa mem -t $SLURM_NTASKS \
  $DIR/$GENOME \
  $DIR/$SAMPLE_R1_001.fastq.gz \
  $DIR/$SAMPLE_R2_001.fastq.gz \
  > $SAMPLE_aln-pe.sam
```

The bwa mem command takes 5 arguments:

- threads
- genome
- fastq files (1+2)
- output file

```
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

export DIR=/projects/e32559/example_data
export SAMPLE=TRIN_5690_11_S31_L001
export GENOME=oenotheraHarringtonii.softmasked.fasta

bwa mem -t $SLURM_NTASKS \
  $DIR/$GENOME \
  $DIR/$SAMPLE_R1_001.fastq.gz \
  $DIR/$SAMPLE_R2_001.fastq.gz \
  > $SAMPLE_aln-pe.sam
```

All the hard coding is gone! Let's do some more practice...

Practice

1. Save your updated script as bwa_vars.sh.
2. Exit your text editor.
3. Copy /projects/e32559/example_data/star.sh to your folder.
4. Open your copy of star.sh to edit.
5. Replace any hard-coding with variables.

What are job arrays?

- Job arrays offer a mechanism for submitting and managing collections of similar jobs quickly and easily, useful for repetitive workloads that follow a **common job pattern**.
- All jobs have the resource options (e.g., time limit, RAM, cores).
- https://slurm.schedmd.com/job_array.html
- https://rcdsdocs.it.northwestern.edu/systems/quest/user-guide/slurm/slurm.html#special-types-of-job-submissions

Job arrays are preferred over loops

could write a for loop in bash that would work through your list of samples

job arrays are an alternative built into SLURM

easier on the scheduling system because they take up less room in the queue while pending

```
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

export DIR=/projects/e32559/example_data
export SAMPLE=TRIN_5690_11_S31_L001
export GENOME=oenotheraHarringtonii.softmasked.fasta

bwa mem -t $SLURM_NTASKS \
  $DIR/$GENOME \
  $DIR/$SAMPLE_R1_001.fastq.gz \
  $DIR/$SAMPLE_R2_001.fastq.gz \
  > $SAMPLE_aln-pe.sam
```

What is the piece that needs to change between samples?

```
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1

module load bwa/0.7.17

export DIR=/projects/e32559/example_data
export SAMPLE=TRIN_5690_11_S31_L001
export GENOME=oenotheraHarringtonii.softmasked.fasta

bwa mem -t $SLURM_NTASKS \
  $DIR/$GENOME \
  $DIR/$SAMPLE_R1_001.fastq.gz \
  $DIR/$SAMPLE_R2_001.fastq.gz \
  > $SAMPLE_aln-pe.sam
```

What is the piece that needs to change between samples?

```
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1
#SBATCH --array=0-66

module load bwa/0.7.17

DIR=/projects/e32559/example_data
GENOME=oenotheraHarringtonii.softmasked.fasta

# read in list of samples
IFS=$'\n' read -d '' -r -a lines < $DIR/list_of_samples.txt

bwa mem -t $SLURM_PROCS \
  $SAMPLE_DIR/$REF_GENOME \
  $DIR/${lines[$SLURM_ARRAY_TASK_ID]}_R1_001.fastq.gz \
  $DIR/${lines[$SLURM_ARRAY_TASK_ID]}_R2_001.fastq.gz > \
  ${lines[$SLURM_ARRAY_TASK_ID]}_aln-pe.sam
```

Use a file with the arguments for each job and iterate over it. In this case, the file is a list of samples.

```
# read in list of samples
IFS=$'\n' read -d '' -r -a lines < $DIR/list_of_samples.txt
```

1. take a list of samples

```
bwa mem -t $SLURM_PROCS \
  $DIR/$REF_GENOME \
  $DIR/${lines[$SLURM_ARRAY_TASK_ID]}_R1_001.fastq.gz \
  $DIR/${lines[$SLURM_ARRAY_TASK_ID]}_R2_001.fastq.gz > \
  ${lines[$SLURM_ARRAY_TASK_ID]}_aln-pe.sam
```

```
# read in list of samples
IFS=$'\n' read -d '' -r -a lines < $DIR/list_of_samples.txt
```

1. take a list of samples

2. read it line by line into a variable called ‘lines’

```
bwa mem -t $SLURM_PROCS \
 $DIR/$REF_GENOME \
 $DIR/${lines[$SLURM_ARRAY_TASK_ID]}_R1_001.fastq.gz \
 $DIR/${lines[$SLURM_ARRAY_TASK_ID]}_R2_001.fastq.gz > \
 ${lines[$SLURM_ARRAY_TASK_ID]}_aln-pe.sam
```

```
# read in list of samples
IFS=$'\n' read -d '' -r -a lines < $DIR/list_of_samples.txt
```

1. take a list of samples

2. read it line by line into a variable called ‘lines’

```
bwa mem -t $SLURM_PROCS \
 $DIR/$REF_GENOME \
 $DIR/${lines[$SLURM_ARRAY_TASK_ID]}_R1_001.fastq.gz \
 $DIR/${lines[$SLURM_ARRAY_TASK_ID]}_R2_001.fastq.gz > \
 ${lines[$SLURM_ARRAY_TASK_ID]}_aln-pe.sam
```

Replace sample ID with line from list
Indexing by iteration in the array

```
#!/bin/bash
#SBATCH --account=e32559
#SBATCH --partition=normal
#SBATCH --job-name=bwa_align
#SBATCH --time=10:00:00
#SBATCH --mem=10G
#SBATCH --ntasks=8
#SBATCH --nodes=1
#SBATCH --array=0-2

module load bwa/0.7.17
SAMPLE_DIR=/projects/e32559/example_data
REF_GENOME=oenotheraHarringtonii.softmasked.fasta

# read in list of samples
IFS=$'\n' read -d '' -r -a lines < $SAMPLE_DIR/list_of_samples.txt

bwa mem -t $SLURM_PROCS $SAMPLE_DIR/$REF_GENOME \
  $SAMPLE_DIR/${lines[$SLURM_ARRAY_TASK_ID]}_R1_001.fastq.gz \
  $SAMPLE_DIR/${lines[$SLURM_ARRAY_TASK_ID]}_R2_001.fastq.gz > ${lines[$SLURM_ARRAY_TASK_ID]}_aln-pe.sam
```

Define an array with the number of samples you have to go through

Let’s launch it!

1. Save your updated script and exit your text editor.
2. Launch the script with `sbatch <script>`
3. Check on the queue with `squeue -u <netid>`

Convert star.sh to an array

1. Reopen star.sh.
2. Use the file /projects/e32559/example_data/star_files.txt to iterate through to convert this script to an array.
3. Don’t forget to add an array sbatch parameter.
4. Launch the script.

Optimize resources for single job before launching an array!

Check Memory Utilization: `seff`

Useful commands:

- ”checkjob” – detailed summary
- “seff” – efficiency summary

```
$ seff 8470584
--------------------------------------------------------------
Job ID: 8470584
Cluster: quest
User/Group: <netid>/<netid>
State: COMPLETED (exit code 0)
Nodes: 1
Cores per node: 16
CPU Utilized: 01:04:40
CPU Efficiency: 93.27% of 01:09:20 core-walltime
Job Wall-clock time: 00:04:20
Memory Utilized: 9.76 GB
Memory Efficiency: 30.49% of 32.00 GB
```

Check Runtimes and CPU Efficiency

```
$ sacct -X --format=reqcpus,reqmem,state,elapsed,jobid
 ReqCPUS      ReqMem       State    Elapsed JobID
--------- ----------- ----------- ----------- -------------
     1         32G   COMPLETED   01:02:18 8470530
     4         32G   COMPLETED   00:16:26 8470533
     8         32G   COMPLETED   00:08:18 8470582
     16        32G   COMPLETED   00:04:20 8470584
```

Check Memory per CPU

1 core, 32G

```
$ seff 8470584
----------------------------------------
Memory Utilized: 2.62 GB
Memory Efficiency: 8.18% of 32.00 GB
```

4 core, 32G

```
$ seff 8470584
----------------------------------------
Memory Utilized: 4.14 GB
Memory Efficiency: 12.95% of 32.00 GB
```

8 cores, 32G

```
$ seff 8470584
----------------------------------------
Memory Utilized: 6.08 GB
Memory Efficiency: 18.99% of 32.00 GB
```

16 cores, 32G

```
$ seff 8470584
----------------------------------------
Memory Utilized: 9.76 GB
Memory Efficiency: 30.49% of 32.00 GB
```